

StackSprint

Development Environment and Project Proposal Plan

CEN3031 - LeetCode Productivity App

September 15, 2026

Team: Santiago Ramirez, Matias Camaran, and Hussain.

Proposal status: StackSprint is the proposed team name. Santiago leads frontend development as requested; the remaining role assignments and technology selections below are proposed for team agreement.

1. Problem and proposed solution

Students preparing for technical interviews often practice inconsistently, have difficulty comparing progress with friends, and track activity across separate tools. StackSprint proposes a web application that combines friendly competition, progress visibility, and solution discussions to support a consistent coding-practice routine.

The solution follows the existing repository concept: weekly leaderboards, discussions that unlock after a problem is solved, and a dashboard combining practice and development activity. The intended users are students and early-career developers preparing for technical interviews.

Challenge alignment: This proposal interprets the challenge as improving motivation and visibility during technical interview preparation. The separate instructor challenge statement was not provided; the team must verify this interpretation against that statement before submission.

2. Project vision

For students and early-career developers **who** need motivation and a clear view of their technical interview preparation, **StackSprint** is a **social coding-practice companion that** combines weekly competition, progress tracking, and discussions unlocked by verified problem completion. **Unlike** practicing alone with separate activity trackers, **our product** brings peer accountability and learning progress into one shared experience.

This vision uses the common Geoffrey Moore positioning structure. The team should compare its wording with the exact template in Chapter 1 of the assigned textbook, which was not supplied.

3. Team roles and collaboration

Member	Primary responsibility
Santiago Ramirez	Frontend lead: interface design, React components, dashboard and leaderboard pages, accessibility, and frontend integration.
Matias Camaran	Proposed project coordinator and backend lead: scope, task tracking, API design, database model, authentication, and release coordination.
Hussain	Proposed integrations and quality lead: data-source feasibility, activity connectors, automated tests, integration checks, and risk monitoring.

All three members implement features, update assigned tasks, document decisions, and review pull requests written by another member. Role ownership establishes accountability without making any component dependent on one person.

4. Scope and success criteria

Feature	Minimum deliverable and acceptance criteria
Profiles and groups	A user can sign in and join a practice group. Group activity and discussion access are checked on the server.
Weekly leaderboard	Rank members by distinct verified problems completed in a defined week. Use a documented UTC week boundary; repeat submissions do not increase the count, and equal scores share a rank.
Unlockable discussions	A user without verified completion cannot read or post to that problem discussion. Access checks apply to API requests as well as the interface. An automated test covers both permitted and denied access.
Activity dashboard	Display completed-problem counts, weekly history, and source timestamps. Keep GitHub events and IDE time separate from solved-problem scores. Show empty, loading, and failed-sync states.
External integrations	Investigate LeetCode verification first. Add a GitHub activity connector after core workflows work; treat WakaTime integration as a stretch feature. Clearly label demo fixtures and never represent them as live verification.

Out of scope: executing untrusted submitted code, recreating LeetCode problem content, building a complete online judge, native mobile applications, and public global competitions. The first release targets small practice groups.

Definition of done: acceptance criteria are met, relevant tests pass, another member reviews the change, documentation is updated, and the approved pull request is merged. A final demonstration must exercise sign-in, leaderboard calculation, discussion access control, and dashboard error handling.

5. Proposed programming languages and technologies

Layer	Selection and rationale
Frontend	TypeScript, React, HTML and CSS, with Vite for development/build tooling. Reusable components support the dashboard, rankings, and discussion screens.
Backend	Node.js with TypeScript and Express. A REST API centralizes account, group, scoring, and discussion authorization logic while sharing a language with the frontend.
Data	PostgreSQL and SQL migrations for users, groups, problems, verified completions, weekly scores, and discussion posts. Constraints prevent duplicate completion records.
Testing and delivery	Vitest for application tests; Playwright for critical browser flows. GitHub Actions will run build and tests after the application scaffold exists. Git and GitHub manage changes; GitHub Projects tracks tasks.
Integration approach	Source-specific adapters normalize external data. Tokens stay on the server; local configuration uses ignored environment files and a committed example containing placeholders only.

These are proposed choices, not claims that the application or its dependencies are already installed. Exact runtime and dependency versions will be selected and locked during scaffolding. The deployment host will be chosen within the course budget and requirements.

6. Development and configuration management

Repository: <https://github.com/Santi852/Cen3031-Proj-1>

Git is the source of truth for code, documentation, migrations, dependency manifests, and lockfiles. Main is the integration branch and serves the role called "Master Branch" in the assignment. Team members work on short-lived feature, fix, or documentation branches and merge through reviewed pull requests.

Control	Team practice
Change workflow	Create or select an issue, branch from current main, make a focused change, verify it locally, and open a pull request linked to the issue. Obtain at least one approval from another teammate.
Branch protection	The live repository reports main as protected. The supplied settings screenshot shows required pull requests, one approval, and dismissal of stale approvals. The repository owner must verify the detailed saved settings; they were not readable through the current contributor account.
Environment consistency	Document setup commands once the scaffold exists. Commit lockfiles and migrations. Keep real secrets out of Git; provide a safe environment-variable example. Review dependency and schema changes through pull requests.
Review and recovery	Resolve review comments before merging. Re-review changed code when approval becomes stale. Revert a problematic merged change through a new pull request; use version tags for agreed course milestones.
Quality checks	Use manual checks for planning documents. Add automated build, type checking, and relevant tests when runnable code exists, then ask the owner to make passing checks required.
Task management	Use Backlog, Ready, In progress, In review, and Done on the existing GitHub project. Every task has an owner and acceptance criteria. Move a task to Done only after its deliverable is verified.

7. Delivery plan

Stage	Deliverable	Lead
1 - Definition	Confirm challenge, scope, roles, stack, integration feasibility, and proposal.	Matias; all review
2 - Foundation	Frontend and backend scaffold, database schema, sign-in, group model, and test workflow.	Santiago / Matias
3 - Core workflows	Leaderboard, completion verification, discussion gates, and basic dashboard.	All members
4 - Integration and release	Source connectors, failure handling, end-to-end tests, documentation, and demonstration.	Hussain; all support

These stages describe sequence, not invented course deadlines. The team will assign dates from the course calendar, review progress twice per week, and raise blockers within 24 hours. Matias will coordinate reprioritization when scope or availability changes.

8. Risk management process

Identify: Review requirements, dependencies, workload, data handling, and integration assumptions at kickoff and whenever scope changes. Record each risk with an owner and an observable warning sign.

Assess and prioritize: Score likelihood and impact from 1 (low) to 3 (high). Multiply them to obtain a priority score: 1-2 low, 3-4 medium, and 6-9 high. Scores are initial team estimates, not measured probabilities.

Respond: Reduce high risks early, avoid work that cannot meet course constraints, and define fallback behavior before depending on external services. Allocate integration research before committing to the complete feature set.

Monitor: Review the register at each twice-weekly check-in. Owners report evidence, current score, mitigation progress, and whether a trigger has occurred. If triggered, open or update an issue, execute the contingency, and revise scope with the team. Matias coordinates escalation to the instructor when the challenge itself is affected.

Risk register: feasibility and delivery

R1 - Unavailable or changing LeetCode data

Priority: $3 \times 3 = 9$ / High. **Owner:** Hussain. **Initial status:** Open.

Trigger: Early feasibility research cannot reliably verify completed problems, or source access fails.

Mitigation: Investigate permitted data access before feature development; isolate the adapter; deduplicate records; retain timestamps and handle rate limits.

Contingency: Use clearly labeled fixtures for the prototype and seek instructor agreement on reduced scope or an approved verification method. Do not unlock real-user discussions from self-reported data.

R2 - Scope exceeds available time

Priority: $3 \times 3 = 9$ / High. **Owner:** Matias. **Initial status:** Open.

Trigger: A core feature slips a planned checkpoint or the estimate exceeds remaining capacity.

Mitigation: Prioritize the core flows and define acceptance criteria. Keep WakaTime and advanced analytics outside the minimum release.

Contingency: Drop stretch integrations, simplify the UI, and deliver a tested core workflow with the reduced scope documented.

R3 - Teammate unavailability or skill gaps

Priority: $2 \times 3 = 6$ / High. **Owner:** Matias. **Initial status:** Open.

Trigger: A blocker remains unresolved for more than 24 hours or an owner misses a checkpoint.

Mitigation: Share setup notes, pair on unfamiliar work, and keep each change small enough for another teammate to review.

Contingency: Reassign the blocked task, pair with the available teammate, and reduce scope if capacity is lower than planned.

R4 - Merge conflicts and inconsistent environments

Priority: $2 \times 2 = 4$ / Medium. **Owner:** Santiago. **Initial status:** Open.

Trigger: Branches diverge or another teammate cannot reproduce setup/build behavior.

Mitigation: Use short-lived branches, reviewed changes, documented setup, lockfiles, and agreed runtime versions.

Contingency: Resolve conflicts with the affected owners and revert a breaking merge if necessary; repair setup notes before continuing.

Risk register: data and product quality

R5 - Unauthorized access or exposed credentials

Priority: $2 \times 3 = 6$ / High. **Owner:** Matias. **Initial status:** Open.

Trigger: An access-control test fails or a real token appears in a commit or log.

Mitigation: Check discussion and group permissions on the server, use least-privilege tokens, ignore local secrets, and review logs and example configuration.

Contingency: Disable affected access, revoke the exposed credential, repair the authorization rule, and rerun the affected tests before restoring the feature.

R6 - Incorrect rankings or duplicate activity

Priority: $2 \times 2 = 4$ / Medium. **Owner:** Hussain. **Initial status:** Open.

Trigger: Repeated submissions increase the score or week-boundary tests produce inconsistent results.

Mitigation: Specify UTC week boundaries, count distinct verified problems, use unique constraints, and test retries and ties.

Contingency: Pause affected rankings, recompute from source completion records, and display a correction notice for affected users.

R7 - External outages or rate limits

Priority: $2 \times 2 = 4$ / Medium. **Owner:** Hussain. **Initial status:** Open.

Trigger: A connector returns repeated failures or a rate-limit response.

Mitigation: Cache recent results, show last-sync time, use bounded retries, and keep core pages usable without an immediate refresh.

Contingency: Display the last successful data as stale and disable refresh temporarily; recover through a controlled resync.

R8 - Late discovery of unusable or broken flows

Priority: $2 \times 2 = 4$ / Medium. **Owner:** Santiago. **Initial status:** Open.

Trigger: A teammate cannot finish the demonstration flow or encounters inaccessible controls.

Mitigation: Review wireframes early; check keyboard access and empty/error states; test the full user journey before release.

Contingency: Prioritize blocking defects, simplify the affected screen, and defer cosmetic improvements.

9. Submission and final review

Submit this proposal as a PDF together with screenshots of the repository, configuration-management documentation, saved branch-protection settings, and a populated project board. Before submission, agree on the proposed team name, remaining roles, and stack; verify challenge alignment and the textbook vision template; and use the course calendar for final task dates.

Reference basis: Assignment instructions and four supplied setup screenshots; repository README inspected September 15, 2026. Technical references: [GitHub protected branches](#) and [React documentation](#). The external integrations remain feasibility assumptions, not verified implementation capabilities.